CODE BASE GUIDE

# Marketing AI Agent Dashboard

How the whole code base works: the parts, how they talk to each other, what every page does, who is allowed to do what, and where the data lives — with flowcharts.

Repository: Marketing_Agent (folders `dashboard/` and `dashboardbackend/`)
Describes the code as of 13 October 2026.


1. What this is
2. The whole system (flowchart)
3. Folder map
4. Signing in and the access rules
5. How pages get their data
6. The pages
7. The agent: Start and Stop
8. Weekly blog files
9. Campaigns
10. Who can do what
11. All routes
12. The database
13. Settings, running and tests
14. Known issues and parked work

## 1. What this is

A company runs a **marketing agent** on an office machine. The agent finds companies that may need its services (*leads*), emails them, reads and classifies their replies, and writes blog posts. Everything it does is stored in a **Supabase** project (a hosted Postgres database with sign-in and file storage).

This repository is the **dashboard** the team uses to watch and steer that agent. It has two programs:

- **Dashboard** ( `dashboard/` ) — a Next.js 15 / React 19 web app. It shows the pages and also has a few small server routes ( `/api/…` ) for things that need Supabase's secret service key: managing users, campaigns and the weekly blog files.
- **Dashboard backend** ( `dashboardbackend/` ) — a Python FastAPI service. It reads the agent's data straight from Postgres (leads, emails, replies, blog posts) and forwards Start / Stop to the agent's control API.

Two rules shape the design:

1. **The dashboard only shows and stores.** Approving emails (in Slack), sending them, publishing blogs and running campaigns are the jobs of other systems. The backend never writes to the agent's tables.
2. **Every request is checked on the server.** The browser shows or hides buttons, but the real rules (signed in, has a role, is an admin) are enforced by the two servers.

> Sign-in is Supabase Auth. Accounts are created by an admin on the Users page — public sign-up is switched off in Supabase, and an account without the `admin` or `employee` role gets no access.

## 2. The whole system

The browser talks to three places: the Next.js app (pages and its `/api` routes), the Python backend (data and the agent), and Supabase Auth (sign-in). Both servers read Supabase. The agent itself runs on the office machine, which nothing on the internet can reach — so Start / Stop goes through a small control API on Render that the agent checks every few seconds.

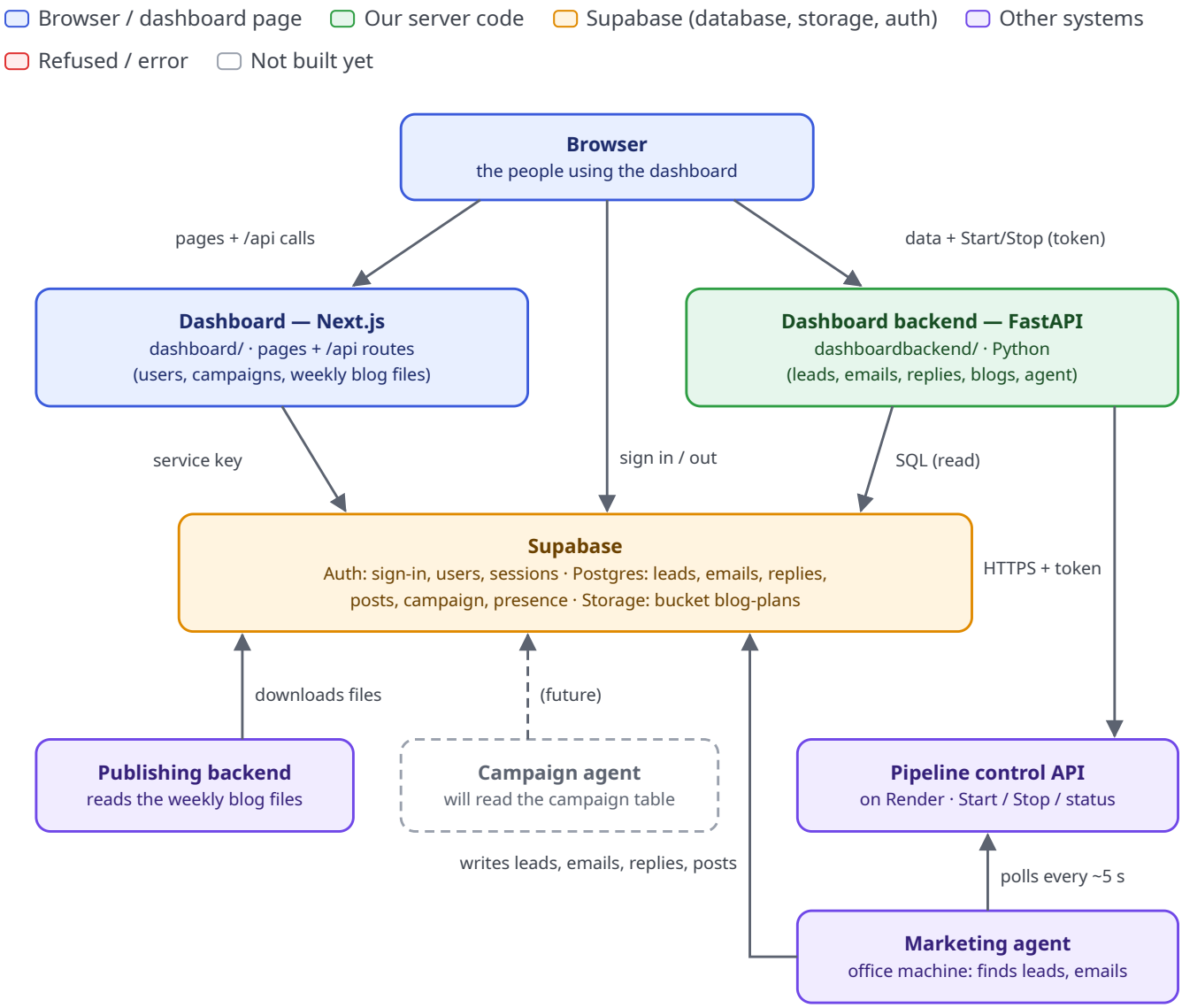



*Figure 1 — The whole system: who talks to whom*

The arrows in short:

- **Browser → Next.js:** loads the pages; calls `/api/…` for users, campaigns, blog files, password.
- **Browser → Python backend:** all lead, email, reply and blog data, and agent Start / Stop / status — always with the user's Supabase token.
- **Next.js → Supabase:** with the service key (server only): user admin, the `campaign` table, the `blog-plans` storage bucket.
- **Python backend → Supabase Postgres:** read-only SQL over a connection pool; it also keeps the small `dashboard_presence` table for "one sign-in per account".
- **Python backend → Pipeline API → Marketing agent:** Start / Stop requests and status.

## 3. Folder map

| Path | What it does |
|---|---|
| `dashboard/app/layout.js` | Root HTML: fonts, sets the light/dark theme before the first paint. |
| `dashboard/app/page.js` | `/` redirects to `/dashboard`. |
| `dashboard/app/login/page.js` | Login page (LoginView). |
| `dashboard/app/(app)/layout.js` | Wraps every signed-in page in AppShell (sign-in guard, sidebar, header). |
| `dashboard/app/(app)/*/page.js` | One tiny file per page: dashboard, history, positive-leads, blogs, campaign, users, profile. |
| `dashboard/app/(app)/error.js` | If a page crashes, shows an error card instead of a blank screen. |
| `dashboard/app/api/…/route.js` | Server routes: account/password, admin/users, blogs/weekly-files, campaigns. |
| `dashboard/components/AppShell.js` | Sign-in guard, sidebar (with the agent box), header (theme, user menu, sign out), 60 s check-in. |
| `dashboard/components/DataProvider.js` | React Query setup: which backend call feeds each section, paging, the leads filter. |
| `dashboard/components/DrawerProvider.js + drawerKinds.js` | The one side drawer that shows a lead, email, reply or blog, with Previous/Next. |
| `dashboard/components/*View.js` | The pages: Dashboard, History, PositiveLeads, Blogs, Campaigns, Users, Profile, Login. |
| `dashboard/components/agent.js` | Agent status wording, the email-mode badge, Start/Stop with the live-email question. |
| `dashboard/components/ui.js, Icon.js, ConfirmDialog.js, FormattedContent.js` | Shared pieces: badges, tables, pager, icons, the in-app confirm box, plain-text formatter. |
| `dashboard/core/supabase.js, auth.js` | The browser's Supabase client; sign in/out, session, authFetch for /api routes. |
| `dashboard/core/api.js` | apiFetch for the Python backend and one function per backend route. |
| `dashboard/core/admin.js, campaigns.js, blogFiles.js` | Browser-side calls to the /api routes. |
| `dashboard/core/weeklyBlogs.js` | Reads and checks a weekly blog JSON (used in the browser and on the server). |
| `dashboard/core/format.js` | Dates, labels, colours for statuses and lead sources. |

| Path | What it does |
| --- | --- |
| `dashboard/core/server/supabaseAdmin.js` | Server only: service-key client, getCaller / requireAdmin, the role rule. |
| `dashboard/core/server/campaigns.js` | Server only: campaign checks, row -> JSON, clear database error messages. |
| `dashboardbackend/app/main.py` | FastAPI app: CORS, routers, /health, /auth/me, /auth/signout. |
| `dashboardbackend/app/auth.py` | Token check, role rule, removed accounts, one sign-in per account, admin check. |
| `dashboardbackend/app/data.py` | The read-only SQL for leads, stats, positive leads, sent emails, replies, blogs. |
| `dashboardbackend/app/agent.py` | Start / Stop / status, forwarded to the pipeline control API. |
| `dashboardbackend/app/blog_plan.py` | Older blog-plan routes (table blog_plan); the page no longer uses them. |
| `dashboardbackend/migrations/001–005` | SQL run by hand in Supabase: presence, blog_plan, campaign, soft delete, roles. |
| `dashboardbackend/tests/` | pytest tests with fake tokens and fake databases (63 tests). |

# 4. Signing in and the access rules

Sign-in happens between the browser and Supabase. Supabase gives the browser an **access token** (a signed JWT, valid about an hour, refreshed automatically). Every call to either server carries it as `Authorization: Bearer …`.

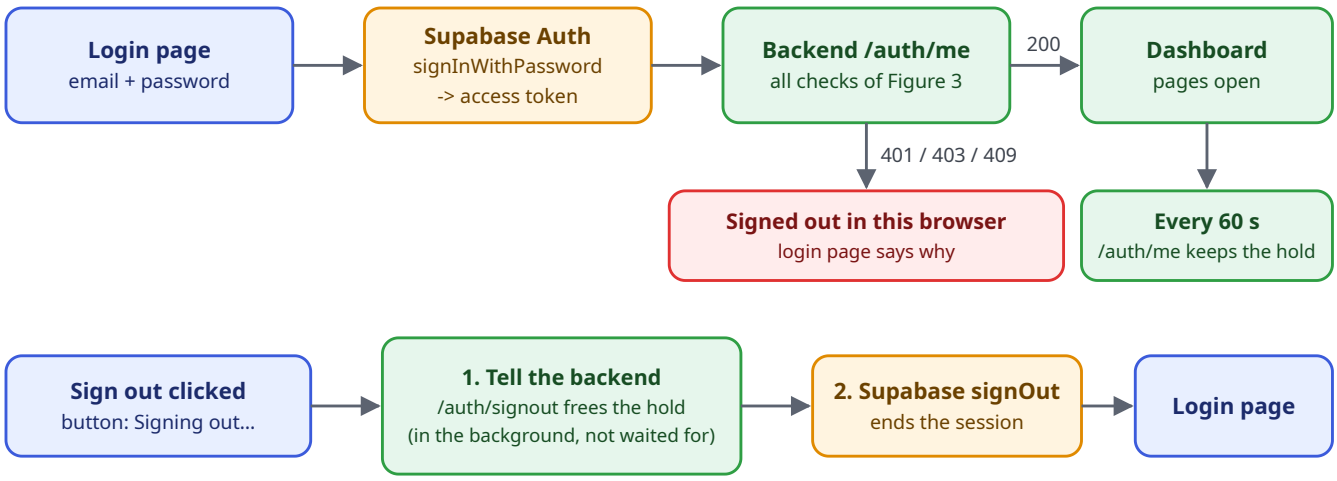



*Figure 2 — Signing in, staying signed in, and signing out*

### One sign-in per account

An account can be used in one place at a time. The backend's `dashboard_presence` table remembers which session holds each account. The open dashboard checks in every 60 seconds. Another device can take the account once the holder signs out, its Supabase session ends, or it has been quiet for 3 minutes (so a closed tab never locks an account for good). A second sign-in meanwhile gets **409** and the message "This account is already signed in on another device."

## What the backend checks on every request

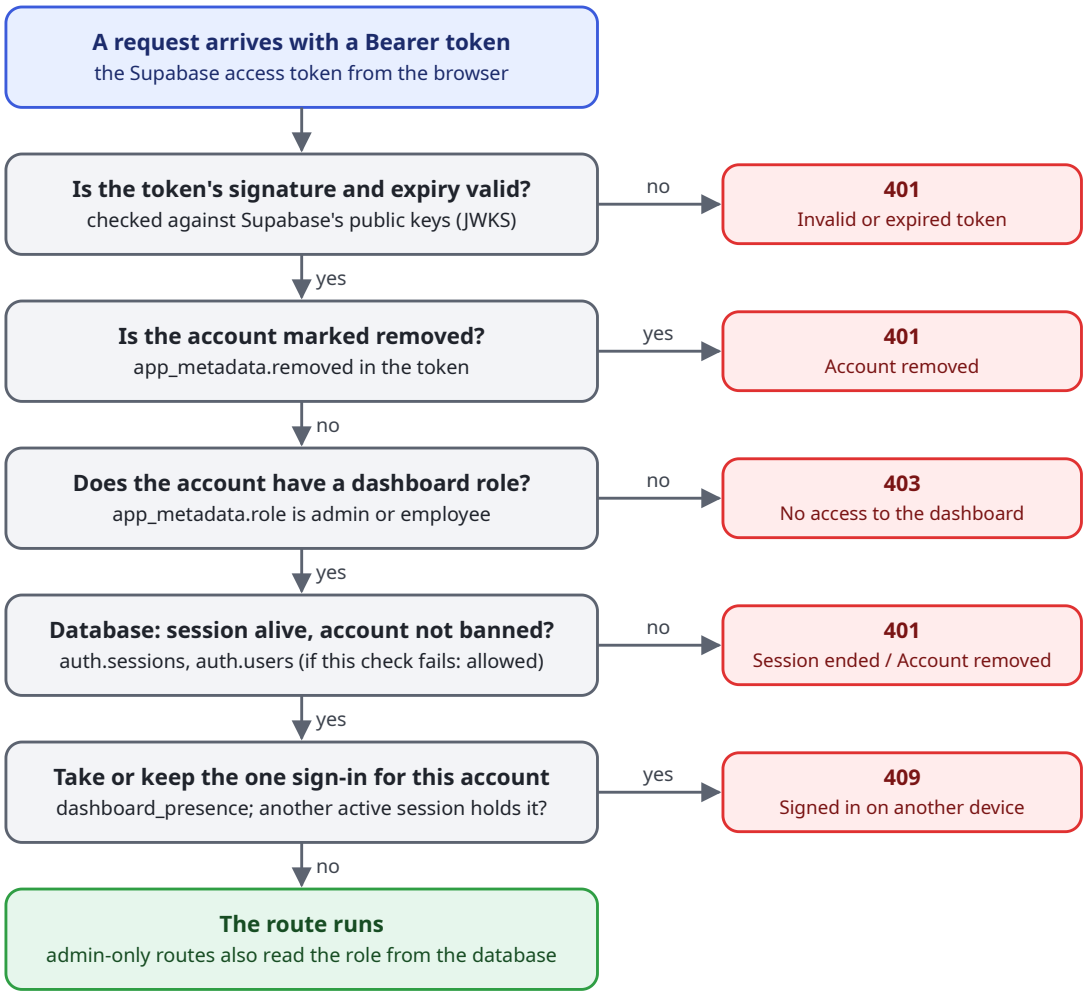



*Figure 3 — What the Python backend checks on every request (app/auth.py, current_user)*

The Next.js `/api` routes do the same role rule in `getCaller` (they ask Supabase for the user behind the token), and `requireAdmin` adds the admin check. Removing a user bans the account for ~100 years and marks it removed; it stays in Supabase.

# 5. How pages get their data

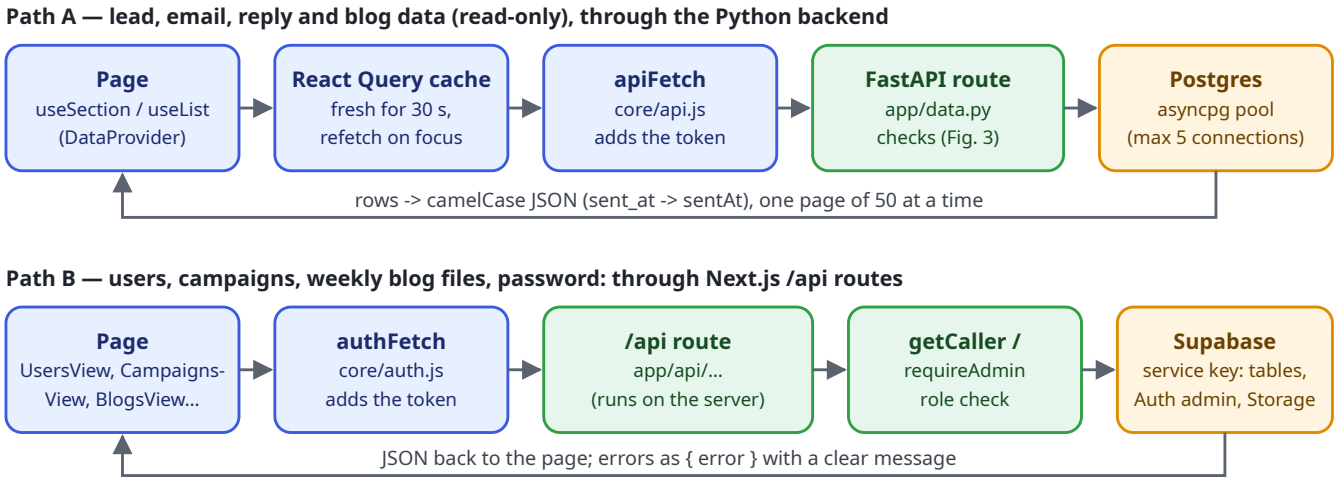



*Figure 4 — How a page gets its data (two paths)*

**Caching.** `DataProvider` wraps the signed-in pages in one React Query cache, so moving between pages reuses data instead of loading it again. Data counts as fresh for 30 seconds and is refreshed in the background when you come back to the tab. Lists come one page (50 rows) at a time; filters and paging run in the database.

**The drawer.** Every list row opens the same side drawer ( `DrawerProvider` ). What it shows for each kind — lead, positive / review / question lead, sent email, reply, blog — is described in `drawerKinds.js` . It loads the full item by id and can step to the previous or next row of the list you were looking at.

### How a lead's status is worked out (backend SQL)

| Status | Rule (first match wins) |
|---|---|
| Responded | The contact has a reply written by a person (not out-of-office, not a bounce). |
| Bounced | Otherwise: an email to them bounced, or a bounce reply came back. |
| Sent (shown as Awaiting) | Otherwise: an email was sent to them. |
| Not Sent | Nothing sent yet. |

A lead's **source** comes from `companies.source` ( `leadgen:feed:hn_hiring` becomes `hn_hiring` , shown as "HN Hiring"). Leads get a stable number, oldest first, shown as #12.

### How Positive Leads are grouped

Each lead is judged on its **latest reply written by a person** (out-of-office and bounces are skipped), so a later "no" replaces an earlier "yes". If the classifier noted low confidence, interested / question / not-interested replies go to **Needs review**; otherwise *interested* → **Positive** and *question* → **Questions**. Each lead also shows whether we answered: replied, queued (approved, not sent yet) or waiting — waiting ones come first.

## 6. The pages

| Page | URL | What it shows / does | Data from |
|---|---|---|---|
| **Dashboard** | `/dashboard` | Greeting; 4 number cards (Total Leads, Leads Emailed, Awaiting Reply, Responses); Lead Pipeline (Found → Emailed → Awaiting → Responded → Bounced); Recent Leads (all leads, newest first, 50 a page); Agent Control; Leads by Source. | /leads/stats, /leads, /agent |
| **History** | `/history` | Two paged lists: emails sent (Delivered / Failed) and replies received. A row opens the email or reply in the drawer. | /emails/sent, /emails/replies |
| **Positive Leads** | `/positive-leads` | Leads judged on their latest reply written by a person, in three groups: Positive (interested), Needs review (classifier unsure), Questions. Waiting-for-our-answer first. Search and sort; the group is kept in the URL. | `/leads/positive` |
| **Blogs** | `/blogs` | Weekly Blog Files: admins upload next week's JSON; everyone sees which blog goes out on which day. Blog Tracker: every blog written so far, Posted / Not Posted. | /api/blogs/weekly-files, /blogs |
| **Campaign** | `/campaign` | Summary (Running now, Upcoming, Total), filters and search, one card per campaign with status and progress. Admins create, edit, switch on/off, delete (soft). | `/api/campaigns` |
| **Users** | `/users` | Admins only: add employees with a password, reset passwords, make admin / remove admin, give access, remove. | `/api/admin/users` |
| **Profile** | `/profile` | Who is signed in, and a form to change your own password. | `/api/account/password` |
| **Login** | `/login` | Email + password. Says why you were signed out (other device, no access). | Supabase Auth, /auth/me |

All signed-in pages share the **AppShell**: a slim sidebar that opens on hover (with the agent's status and Start / Stop), and a header with the light / dark switch and the user menu (Profile, Sign out). On narrow screens the sidebar becomes a row across the top.

## 7. The agent: Start and Stop

The agent runs on the office machine. The dashboard never talks to it directly: the backend records the request in the pipeline control API, the agent picks it up within about 5 seconds (stopping waits for the current step, up to 30 s), and the dashboard keeps asking for the status until it matches.

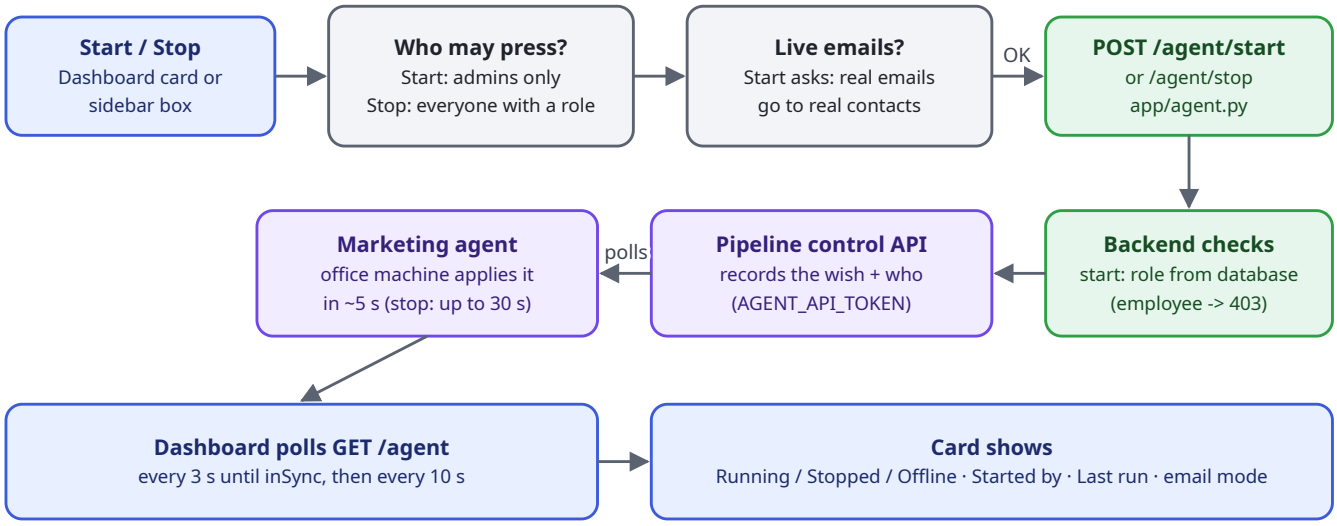



*Figure 5 — Starting and stopping the marketing agent*

> **Email mode.** The card shows whether emails are real: **Live emails** (to the real contacts), **Test mode** (all to a test inbox) or **Sending off**. Starting in live mode asks for confirmation first. Only admins can start; anyone with a role can stop — stopping is the safety brake, and "Stopped by" shows who did it.

## 8. Weekly blog files

Once a week an admin uploads next week's blogs as one JSON file. The dashboard stores it as-is in Supabase Storage; another backend reads it and publishes the blogs. The Blogs page shows the schedule straight from those files, so it always matches what the publishing backend gets.

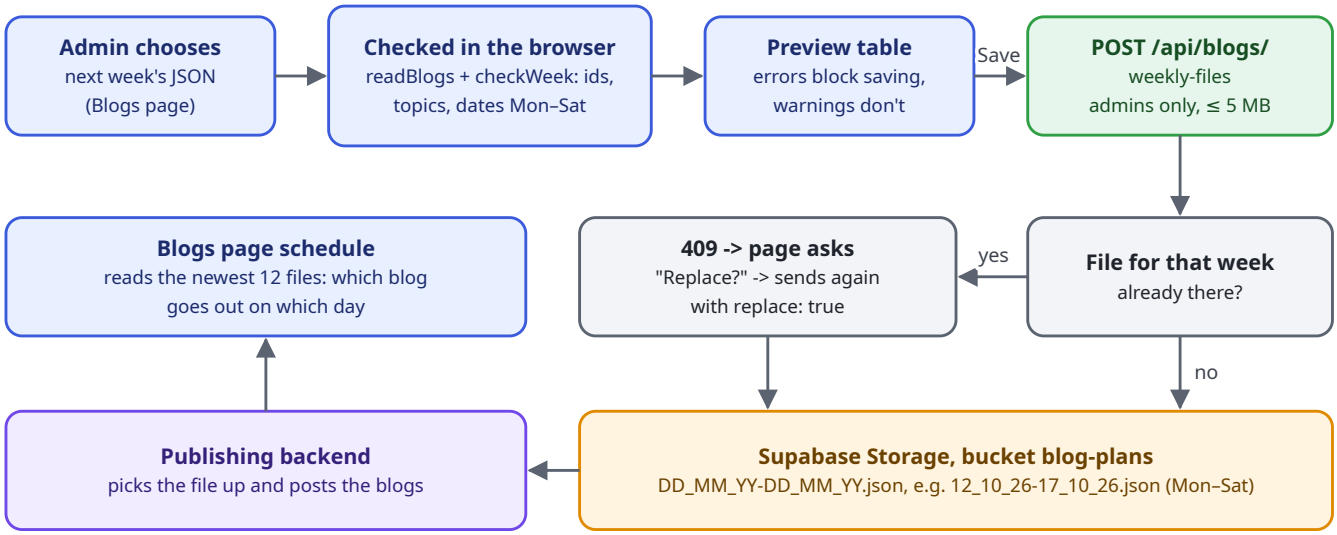
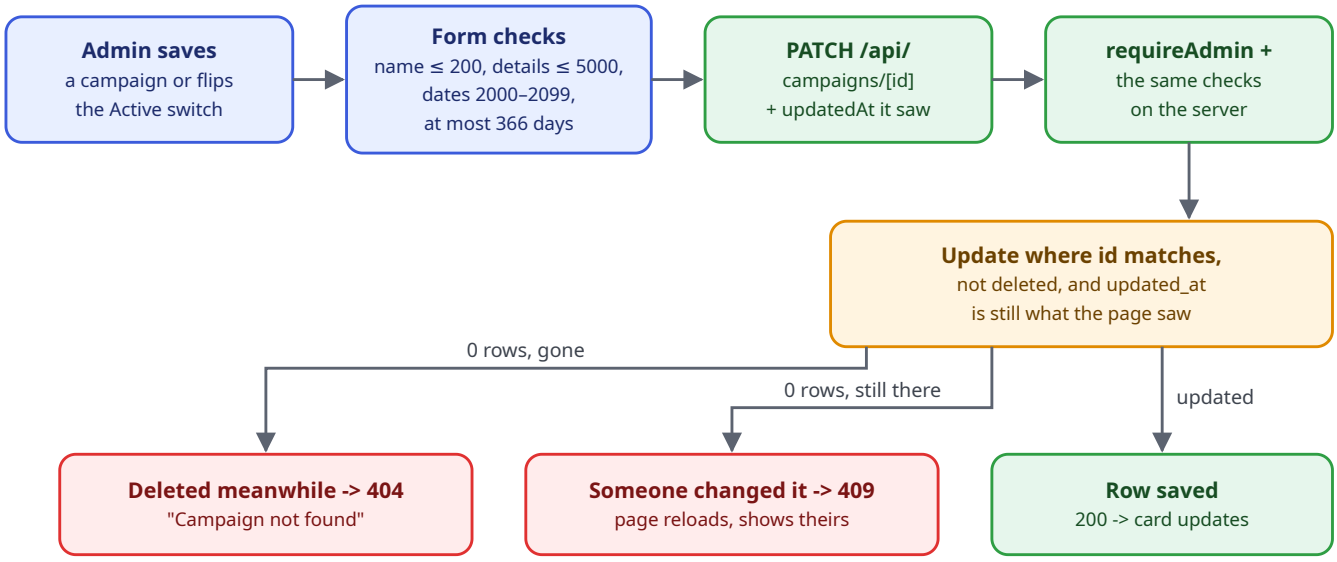



*Figure 6 — Uploading the weekly blog file*

Each blog in the file has: `id`, `topic`, `category`, `keywords`, `publish_date` (inside that Monday–Saturday), `tone`, `length`, `target_versions`, `reference_urls`, `content_format` and `content`. The lower card, **Blog Tracker**, is separate: it lists the posts the agent actually wrote (`content_posts`).

## 9. Campaigns

A campaign has a name, details, a start and end date, and an Active switch. Its status is worked out from today's date: **Running** (with a progress bar and days left), **Upcoming**, **Ended** or **Inactive**. The page goes through the Next.js routes and the service key — the Python backend is not involved.




Delete is a soft delete: deleted_at is set, the row stays in Supabase and is hidden from the page.
Creating a campaign is POST /api/campaigns (admins only, same checks).

*Figure 7 — Saving a campaign (with the "someone else changed it" check)*

Right now nothing outside the page reads the campaign table. The plan (not built) is a campaign agent that picks up new campaigns, writes a plan back, and waits for an admin to approve or reject it on this page.

## 10. Who can do what

| Action | Employee | Admin | Enforced by |
|---|---|---|---|
| See leads, emails, replies, blogs, stats | ✔ | ✔ | Backend: current_user |
| See campaigns and the blog schedule | ✔ | ✔ | Next: getCaller |
| Change own password | ✔ | ✔ | Next: getCaller + current password |
| Stop the agent | ✔ | ✔ | Backend: current_user |
| Start the agent | — | ✔ | Backend: current_admin (role from database) |
| Create / edit / switch / delete campaigns | — | ✔ | Next: requireAdmin |
| Upload the weekly blog file | — | ✔ | Next: requireAdmin |
| Manage users (add, reset, roles, remove) | — | ✔ | Next: requireAdmin; not your own role or account |
| Any account without a role | nothing | | 403 "no access" on both servers |

The admin role lives in `app_metadata.role`, which only the service key can change — a user can't give it to themselves. For starting the agent the backend reads the role from the database, so a demotion takes effect at once.

# 11. All routes

### Python backend ( `dashboardbackend/app` )

| Route | Who | What |
|---|---|---|
| `GET /health` | anyone | Liveness check. |
| `GET /auth/me` | role | Who you are; also refreshes the one-sign-in hold. |
| `POST /auth/signout` | role | Frees the account for another device at once. |
| `GET /leads` | role | One page of leads (filters: status, source). |
| `GET /leads/stats` | role | Counts by status and by source. |
| `GET /leads/positive` | role | Positive / Needs review / Questions groups. |
| `GET /leads/{id}` | role | One lead with its latest reply label. |
| `GET /emails/sent[/{id}]` | role | Sent emails; one with its body. |
| `GET /emails/replies[/{id}]` | role | Replies; one with the email it answers. |
| `GET /blogs[/{id}]` | role | Blog posts (content_posts) and Posted count. |
| `GET /agent` | role | Agent status from the pipeline API, with the requester's name. |
| `POST /agent/stop` | role | Ask the agent to stop. |
| `POST /agent/start` | admin | Ask the agent to start. |
| `/blogs/plan…` | role / admin | Older blog-plan routes; not used by the page any more. |

### Next.js server routes ( `dashboard/app/api` )

| Route | Who | What |
|---|---|---|
| `POST /api/account/password` | role | Checks the current password, sets the new one. |
| `GET /api/admin/users` | admin | List users (removed ones hidden). |
| `POST /api/admin/users` | admin | Create an employee (role employee) with the admin's chosen password. |
| `PATCH /api/admin/users/[id]` | admin | New password, or role admin / employee (not your own). |
| `DELETE /api/admin/users/[id]` | admin | Remove: banned + marked removed; stays in Supabase. |
| `GET /api/blogs/weekly-files[?name=]` | role | List files with their schedule, or one file. |
| `POST /api/blogs/weekly-files` | admin | Save a week's JSON (409 if it exists, unless replace). |
| `GET /api/campaigns` | role | Campaigns not deleted, newest start first. |
| `POST /api/campaigns` | admin | Create a campaign. |
| `PATCH /api/campaigns/[id]` | admin | Change a campaign; refused (409) if someone changed it since. |
| `DELETE /api/campaigns/[id]` | admin | Soft delete (sets deleted_at). |

"role" = signed in with the admin or employee role; anything else gets 401 / 403.

# 12. The database

### Tables the agent writes (the dashboard only reads them)

| Table | Holds |
|---|---|
| `companies` | A company the agent found: name, domain, source, summary, industry. |
| `contacts` | A person at a company. A **lead** = a contact joined to its company. |
| `emails` | Emails the agent drafted and sent (status sent, bounced, approved, sending…). |
| `replies` | Replies received, with the classifier's label (interested, question, not_interested, ooo, bounce…). |
| `content_posts` | Blog posts and where they were published (Dev.to, WordPress, LinkedIn). |
| `auth.users / auth.sessions` | Supabase's own tables: role, removed/banned flags, live sign-in sessions. |

### Tables and storage the dashboard owns

| Name | SQL file (migrations/) | Holds |
|---|---|---|
| `dashboard_presence` | 001 (also created automatically) | Which session is using each account, and when it was last seen. |
| `blog_plan` | 002 (also created automatically) | Older blog briefs; no longer used by the page. |
| `campaign` | 003, 004 | Campaigns; deleted_at for soft delete. |
| `auth.users roles` | 005 | Gives the employee role to existing employees (fill in the emails, run once). |
| `Storage: blog-plans` | created on first use | The weekly blog JSON files (private bucket). |

The SQL files are run by hand in the Supabase SQL editor. Every dashboard-owned table has row-level security switched on with no policies, so the browser's public key can't read or write them — only the two servers can.

## 13. Settings, running and tests

### Settings (names only — the values are secrets kept in env files, never in git)

| Where | Variables |
|---|---|
| dashboard (.env.local) | NEXT_PUBLIC_SUPABASE_URL, NEXT_PUBLIC_SUPABASE_ANON_KEY (public), NEXT_PUBLIC_API_URL (the Python backend), SUPABASE_SERVICE_ROLE_KEY (server only, secret) |
| dashboardbackend (.env) | SUPABASE_URL, DATABASE_URL (pooled Postgres), CORS_ORIGINS, AGENT_API_TOKEN (or PIPELINE_API_TOKEN), AGENT_API_URL (default: the Render URL), SUPABASE_JWT_SECRET (only for old HS256 projects) |

### Running it

- **Dashboard:** `cd dashboard && npm run dev` (http://localhost:3000). Don't run `npm run build` while the dev server is running — it breaks the dev server's files.
- **Backend:** `cd dashboardbackend && uvicorn app.main:app --reload --port 8000`
- **Tests:** `cd dashboardbackend && pytest` — 63 tests using fake tokens and fake databases; they never touch real data.

### Adding things

- **A new data route:** write the SQL in `app/data.py` (protected automatically by the router), then one `apiFetch()` line in `core/api.js`.
- **A new admin action:** a Next.js route under `app/api/` that starts with `requireAdmin(request)`.
- **A schema change:** a new numbered SQL file in `dashboardbackend/migrations/`, run by hand.

## 14. Known issues and parked work

Found while reviewing the code base; none are urgent security problems.

| Area | What |
|---|---|
| Blogs page | Server does not re-check the weekly file's contents; page downloads up to 12 whole files each visit; the blog viewer finds a blog by position; markdown (code blocks, links) shows as plain text; schedule order is confusing. |
| Dashboard page | Recent Leads hides companies with no contact; agent status is asked every 10 s per open tab; small text issues (greeting, time of day). |
| Campaigns | Nothing reads the campaign table yet (the approval loop with the campaign agent is planned, not built); no record of who created, changed or deleted a campaign. |
| Backend | If the database check fails, a request is allowed (logged); lead queries scan every lead on each load; Python versions are not pinned; SQL migrations are run by hand. |
| Repository | The .next build folder is committed to git; the README mentions files and paths that no longer exist. |
| Accounts | Changing your password creates an extra Supabase session; other devices stay signed in after a password change. |

> Recently fixed: role required on both servers, faster sign-out, agent Start admin-only with a live-email confirmation, campaign edit conflicts, campaign date limits, clearer campaign database errors, and the Dashboard's numbers (bounced leads shown, one base for percentages, "Leads Emailed").